Java Console-Based Banking Application

Submitted By:
Name: Makwana Bhumi B.
Enrollment Number: D26AIML087
Class / Semester: Sem – 3, Batch - E

Submitted To:
Jalpesh Vasa

Department of Artificial Intelligence and Machine Learning
CHARUSAT

Academic Year: 2026–2027

TABLE OF CONTENTS

1. Introduction
2. Objectives
3. Project Overview
4. Software and Hardware Requirements
5. Features of the System
6. System Design and Architecture
7. Concepts of Java Used
8. Implementation and Source Code
9. Output Screenshots
10. Testing
11. Conclusion
12. Learning Outcomes
13. References

1. INTRODUCTION

MiniBank is a Java-based console banking application developed to demonstrate important concepts of Java programming. The application provides basic banking operations such as opening an account, depositing money, withdrawing money, transferring funds, and listing account details.

The project uses object-oriented programming concepts, exception handling, collections, multithreading, file handling, annotations, reflection, and other Java features. It also demonstrates saving account data to a file and generating a banking report.

2. OBJECTIVES

The main objectives of the MiniBank project are:

- To develop a simple console-based banking application.
- To implement object-oriented programming concepts in Java.
- To manage bank accounts and their balances.
- To perform deposit, withdrawal, and fund transfer operations.
- To handle invalid inputs and exceptional situations.
- To demonstrate multithreading and synchronization.
- To store account information using file handling.
- To generate a report containing account details.
- To understand Java collections, generics, annotations, and reflection.

3. PROJECT OVERVIEW

MiniBank is a menu-driven banking application that runs in the Java console. Users can select banking operations from a menu and perform the corresponding tasks.

The application is divided into multiple Java classes, with each class responsible for a particular task.

Main Modules

1. Account Management: Creates and manages bank accounts.
2. Transaction Management: Performs deposits, withdrawals, and transfers.
3. Validation: Checks input data and prevents invalid operations.
4. Exception Handling: Handles errors such as invalid amounts and missing accounts.
5. Multithreading: Demonstrates concurrent banking operations.
6. File Storage: Saves and loads account information.
7. Logging and Reporting: Records events and generates account reports.
8. Collections and Repository: Demonstrates storing and managing objects using Java collections and generics.

4. SOFTWARE AND HARDWARE REQUIREMENTS

Software Requirements

- Programming Language: Java
- Java Development Kit (JDK): JDK 21
- IDE / Editor: Visual Studio Code or another Java-supported editor
- Operating System: Windows
- Terminal: Windows PowerShell

Hardware Requirements

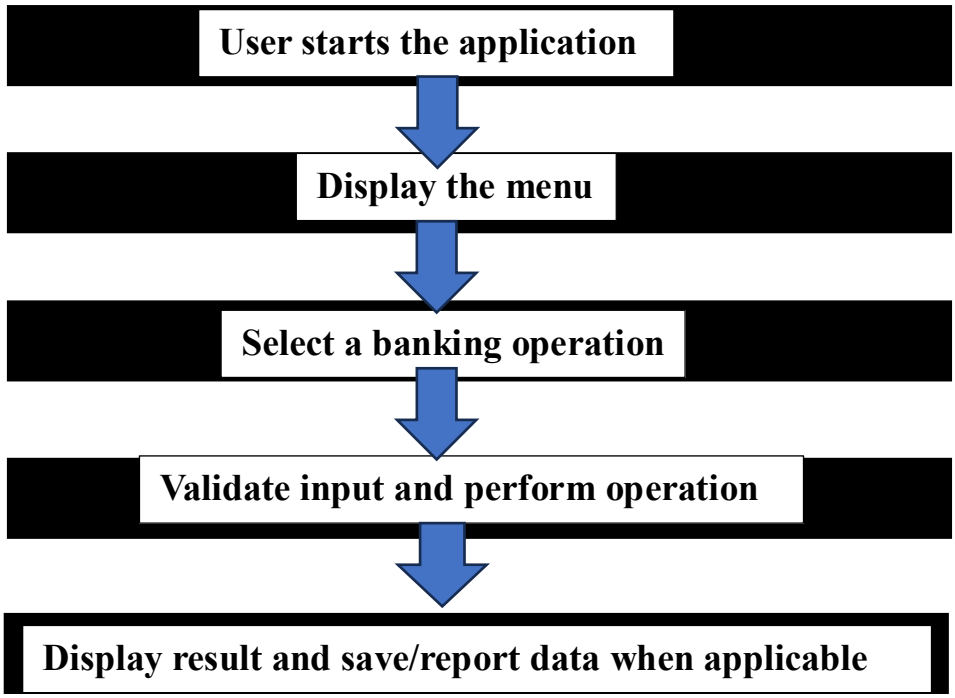
- Computer or laptop
- Minimum 4 GB RAM recommended
- Keyboard and display
- Sufficient storage space for source files and generated reports

5. FEATURES OF THE SYSTEM

The MiniBank application includes the following features:

- Creation of savings accounts.
- Deposit and withdrawal operations.
- Transfer of money between accounts.
- Display of account details and balances.
- Validation of banking operations.
- Custom exception classes for banking errors.
- Thread-safe account balance operations.
- Use of ExecutorService and wait-notify mechanisms for multithreading demonstrations.
- Saving and loading account data through files.
- Logging banking events.
- Generating a text-based account report.
- Demonstrating List, Set, Map, Queue, generics, annotations, and reflection.

6. System Design and Java Concepts



7. SYSTEM DESIGN AND JAVA CONCEPTS

6.1 Object-Oriented Programming

The project uses classes and objects to represent bank accounts and banking services. Inheritance is used to create specialized account types, while abstraction defines common account behaviour.

6.2 Interfaces and Enumerations

Interfaces define the operations supported by the banking service. Enumerations represent a fixed set of transaction types and menu options.

6.3 Exception Handling

Custom exceptions are used to represent banking errors such as invalid amounts, missing accounts, and insufficient balances.

6.4 Multithreading and Synchronization

Threads are used to demonstrate concurrent banking operations. Synchronization helps protect account balance operations from unsafe concurrent access.

6.5 Collections and Generics

Java collections store and manage account objects and other data. Generics allow reusable repository functionality.

6.6 File Handling, Logging, and Reporting

File handling is used to save and read account data. Logging records application events, and report generation produces a text file containing account information.

6.7 Annotations and Reflection

Annotations provide metadata for classes and fields. Reflection is used to inspect class information and identify annotated fields.

8. Implementation and source code

8.1 Main Program — MiniBank.java

Purpose:

The MiniBank class provides the main menu of the banking application and allows users to select different banking operations.

Source Code:

```
import java.util.Scanner;

public class MiniBank {

    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        BankService bankService = new BankService();

        BankInfo bank = new BankInfo(
            name: "MiniBank",
            branch: "Anand Branch"
        );

        System.out.println("================================");
        System.out.println("       " + bank.name());
        System.out.println("       " + bank.branch());
        System.out.println("================================");

        boolean running = true;

        while (running) {
            try {
                System.out.println("\n===== MAIN MENU =====");
                System.out.println("1. Open Account");
                System.out.println("2. Deposit");
                System.out.println("3. Withdraw");
                System.out.println("4. Transfer");
                System.out.println("5. List Accounts");
                System.out.println("6. Exit");
                System.out.print("Enter your choice: ");
```

```
                if (!sc.hasNextInt()) {
                    System.out.println("Please enter a valid number.");
                    sc.nextLine();
                    continue;
                }

                int choice = sc.nextInt();
                sc.nextLine();
```

```
switch (choice) {
    case 1 -> {
        System.out.print("Enter customer name: ");
        String name = sc.nextLine();

        System.out.print("Enter opening balance: ");
        long amount = sc.nextLong();
        sc.nextLine();

        Account account =
                bankService.openAccount(name, amount);

        System.out.println("Account created successfully!");
        System.out.println(account);
    }

    case 2 -> {
        System.out.print("Enter account number: ");
        String number = sc.nextLine();

        System.out.print("Enter deposit amount: ");
        long amount = sc.nextLong();
        sc.nextLine();

        bankService.deposit(number, amount);
        System.out.println("Deposit successful!");
    }
```

```
    case 3 -> {
        System.out.print("Enter account number: ");
        String number = sc.nextLine();

        System.out.print("Enter withdrawal amount: ");
        long amount = sc.nextLong();
        sc.nextLine();

        bankService.withdraw(number, amount);
        System.out.println("Withdrawal successful!");
    }

    case 4 -> {
        System.out.print("Enter sender account number: ");
        String from = sc.nextLine();

        System.out.print("Enter receiver account number: ");
        String to = sc.nextLine();

        System.out.print("Enter transfer amount: ");
        long amount = sc.nextLong();
        sc.nextLine();

        bankService.transfer(from, to, amount);
        System.out.println("Transfer successful!");
    }

    case 5 -> bankService.listAccounts();
```

```
                case 6 -> {
                    System.out.println(
                        "Thank you for using MiniBank!"
                    );
                    running = false;
                }

                default -> System.out.println(
                    "Invalid choice. Please try again."
                );
            }

        } catch (IllegalArgumentException |
                 IllegalStateException e) {
            System.out.println("Error: " + e.getMessage());

        } catch (ArithmeticException e) {
            System.out.println("Error: Amount is too large.");

        } catch (java.util.InputMismatchException e) {
            System.out.println("Error: Please enter a valid amount.");
            sc.nextLine();
        }
    }

    sc.close();
  }
}
```

8.2 Account.java — Account Management

```
import java.util.Objects;
public abstract class Account {

    private final String accountNumber;
    private String ownerName;
    private long balance;
    private boolean active;

    private static int accountCounter = 0;

    public Account(String ownerName, long openingBalance) {
        if (openingBalance < 0) {
            throw new IllegalArgumentException(
                "Opening balance cannot be negative."
            );
        }
        this.accountNumber =
                String.format("AC%04d", ++accountCounter);

        this.ownerName = ownerName;
        this.balance = openingBalance;
        this.active = true;
    }
```

```java
public Account(String ownerName) {
    this(ownerName, openingBalance: 0);
}

public abstract double interestRate();

public abstract boolean canWithdraw(long amount);

public String getAccountNumber() {
    return accountNumber;
}
```

```java
public String getOwnerName() {
    return ownerName;
}

public synchronized long getBalance() {
    return balance;
}

public boolean isActive() {
    return active;
}

public synchronized void deposit(long amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException(
            "Deposit amount must be positive."
        );
    }

    if (!active) {
        throw new IllegalStateException("Account is inactive.");
    }

    balance = Math.addExact(balance, amount);
}

public synchronized boolean withdraw(long amount) {
    if (amount <= 0) {
        throw new IllegalArgumentException(
            "Withdrawal amount must be positive."
        );
    }
```

```java
    if (!active || !canWithdraw(amount)) {
        return false;
    }

    balance = Math.subtractExact(balance, amount);
    return true;
}
```

```
    @Override
    public String toString() {
        return "Account Number: " + accountNumber
                + " | Owner: " + ownerName
                + " | Balance: Rs. " + getBalance()
                + " | Interest: " + interestRate() + "%"
                + " | Active: " + active;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }

        if (!(obj instanceof Account)) {
            return false;
        }

        Account other = (Account) obj;
        return accountNumber.equals(other.accountNumber);
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNumber);
    }
}
```

8.3 SavingsAccount.java — Inheritance

```
public class SavingsAccount extends Account {

    private long minBalance;
    public SavingsAccount(
            String ownerName,
            long openingBalance,
            long minBalance) {

        super(ownerName, openingBalance);

        if (minBalance < 0 || minBalance > openingBalance) {
            throw new IllegalArgumentException(
                "Minimum balance must be between 0 and opening balance."
            );
        }
        this.minBalance = minBalance;
    }
    public SavingsAccount(String ownerName, long openingBalance) {
        this(ownerName, openingBalance, minBalance: 0);
    }
```

```
    @Override
    public double interestRate() {
        return 4.0;
    }
    @Override
    public boolean canWithdraw(long amount) {
        return amount > 0
                && getBalance() >= minBalance
                && amount <= getBalance() - minBalance;
    }
    public long getMinBalance() {
        return minBalance;
    }
}
```

8.4 BankService.java — Banking Operations

```
public Account openAccount(String name, long openingBalance) {

    if (name == null || name.trim().isEmpty()) {
        throw new IllegalArgumentException(
            "Name cannot be empty."
        );
    }

    if (openingBalance < 0) {
        throw new IllegalArgumentException(
            "Opening balance cannot be negative."
        );
    }

    Account account =
        new SavingsAccount(name, openingBalance, minBalance: 0);

    accounts.add(account);

    return account;
}
```

```
public Account findAccount(String accountNumber) {

    for (Account account : accounts) {
        if (account.getAccountNumber().equals(accountNumber)) {
            return account;
        }
    }

    return null;
}
```

```java
    @Override
    public boolean deposit(String accountNumber, long amount) {

        Account account = findAccount(accountNumber);

        if (account == null || amount <= 0) {
            return false;
        }

        account.deposit(amount);
        return true;
    }

    // Normal withdrawal method
    @Override
    public boolean withdraw(String accountNumber, long amount) {

        Account account = findAccount(accountNumber);

        if (account == null || amount <= 0) {
            return false;
        }

        return account.withdraw(amount);
    }
```

```java
    @Override
    public boolean transfer(
            String fromAccount,
            String toAccount,
            long amount) {

        if (amount <= 0) {
            return false;
        }

        Account source = findAccount(fromAccount);
        Account destination = findAccount(toAccount);

        if (source == null || destination == null) {
            return false;
        }

        if (source == destination) {
            return false;
        }

        if (!source.withdraw(amount)) {
            return false;
        }

        try {
            destination.deposit(amount);
            return true;
```

```
        } catch (RuntimeException e) {
            source.deposit(amount);
            return false;
        }
    }

```

```
    public List<Account> listAccounts() {
        return new ArrayList<>(accounts);
    }
```

8.5 BankFileStorage.java — File Handling

```
    public static void saveAccounts(List<Account> accounts)
            throws IOException {

        try (BufferedWriter writer =
                     new BufferedWriter(new FileWriter(FILE_NAME))) {

            for (Account account : accounts) {
                String name = account.getOwnerName();
                name = name.replace("|", " ");

                writer.write(name + "|" + account.getBalance());
                writer.newLine();
            }
        }

        System.out.println("Account data saved successfully.");
    }
```

```
    public static void readAccounts() throws IOException {
        try (BufferedReader reader =
                     new BufferedReader(new FileReader(FILE_NAME))) {

            String line;

            System.out.println("\n--- Saved Account Data ---");

            while ((line = reader.readLine()) != null) {
                String[] parts = line.split("\\|", 2);

                if (parts.length == 2) {
                    System.out.println(
                        "Name: " + parts[0]
                        + ", Balance: " + parts[1]
                    );
                }
            }
        }
    }
```

```
    public static void loadAccounts(BankService bank)
            throws IOException {

        try (BufferedReader reader =
                    new BufferedReader(new FileReader(FILE_NAME))) {

            String line;

            while ((line = reader.readLine()) != null) {
                String[] parts = line.split("\\|", 2);

                if (parts.length == 2) {
                    try {
                        String name = parts[0];
                        long balance = Long.parseLong(parts[1]);

                        if (!name.isBlank() && balance >= 0) {
                            bank.openAccount(name, balance);
                        }
                    } catch (IllegalArgumentException e) {
                        System.out.println(
                            "Skipping invalid record: " + line
                        );
                    }
                }
            }
        }
```

8.6 BankReport.java — Report Generation

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class BankReport {

    public static void generate(BankService bank)
            throws IOException {

        try (BufferedWriter writer =
                new BufferedWriter(new FileWriter("report.txt"))) {

            writer.write("========== MINIBANK REPORT ==========");
            writer.newLine();
            writer.write("Total Accounts: "
                    + bank.listAccounts().size());
            writer.newLine();
            writer.newLine();

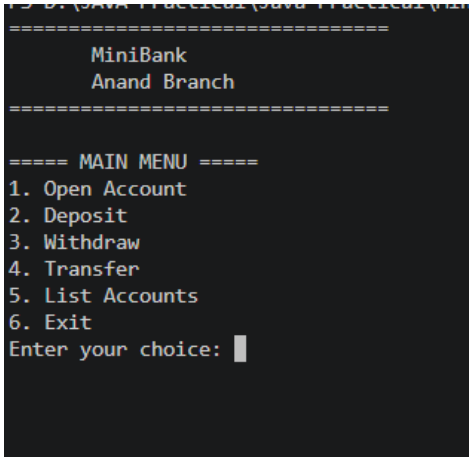
            for (Account account : bank.listAccounts()) {
                writer.write(account.toString());
                writer.newLine();
            }

            writer.write("========== END OF REPORT ==========");
        }

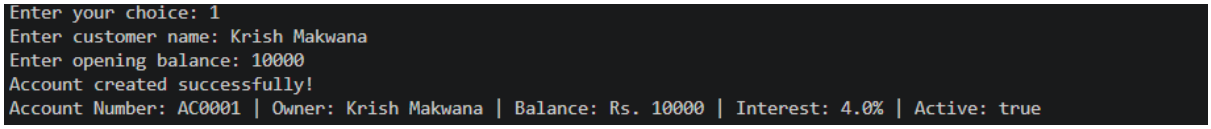
        System.out.println("Report saved to report.txt");
    }
}
```

9.Output screenshots

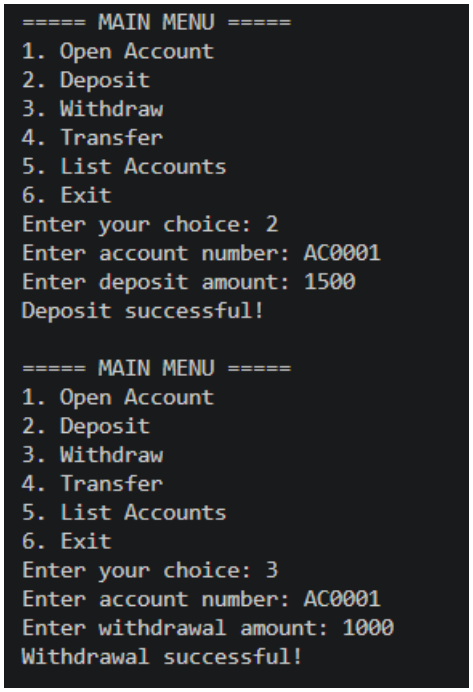
MiniBank Main Menu



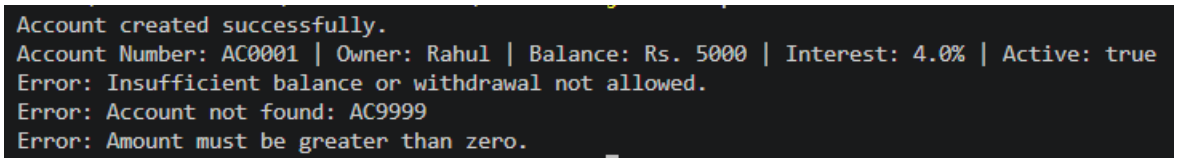
Account Creation



Deposit / Withdrawal



Exception Handling



Multithreading

```
Initial balance: 5000
Thread-3 deposited 500
Thread-2 withdrew 2000
Thread-1 deposited 1000
Final balance: 4500
```

Final Integration and Report

```
Previous data loaded.
Account data saved successfully.
Report saved to report.txt

Final Accounts:
Account Number: AC0001 | Owner: Rahul | Balance: Rs. 7000 | Interest: 4.0% | Active: true
PS D:\JAVA Practical\Java-Practical\MiniBank>
```

```
========== MINIBANK REPORT ==========
Total Accounts: 1

Account Number: AC0001 | Owner: Rahul | Balance: Rs. 7000 | Interest: 4.0% | Active: true
========== END OF REPORT ==========
```

9. TESTING

The application was compiled and executed using JDK 21. Testing was performed to verify the basic functionality of the banking system.

Test Case	**Expected Result**
Create a valid account	Account details are displayed
Deposit a valid amount	Balance increases
Withdraw an allowed amount	Balance decreases
Enter an invalid amount	The operation is rejected or an exception is handled
Save account data	Account data is written to a file
Generate a report	The report file is created

Actual test results should be recorded after running each test case.

10. CONCLUSION

The MiniBank project demonstrates the development of a console-based banking application using Java. It combines basic banking operations with object-oriented programming, exception handling, collections, multithreading, file handling, and report generation.

This project helped demonstrate how Java concepts can be organized into separate classes to build a modular application.

11. LEARNING OUTCOMES

Through this project, I learned:

- How to design classes and objects in Java.
- How to implement inheritance, abstraction, and interfaces.
- How to validate inputs and handle exceptions.
- How to use collections and generics.
- How to work with threads and synchronization.
- How to read and write data using files.
- How to use annotations and reflection.
- How to compile, run, and package a Java application into a JAR file.

12. REFERENCES

1. Java SE Documentation — https://docs.oracle.com/en/java/
2. Class notes and practical instructions provided by the faculty.
3. Source code and implementation files of the MiniBank project.